Dublin Bikes → Spring Boot

练习册

6 关 · 58 个测试 · 边做题边学 Spring Boot

Java 25 · Spring Boot 4.1.2 · Spring Framework 7 · Hibernate 7 · Jackson 3

代码：仓库根目录　答案：`exercises/solutions/`

目录

开始之前 **1**

Part 1 现代 Java：record、Stream、Optional **3**

知识点 3

题目 5

Part 2 Spring 核心：容器、Bean、依赖注入、配置 **8**

知识点 8

题目 10

Part 3 调用外部 API：RestClient 与 JSON 映射 **12**

知识点 12

题目 14

Part 4 数据库：JPA 实体、复合主键、Repository **16**

知识点 16

题目 19

常见报错 21

Part 5 事务和定时任务：@Transactional、@Scheduled、Mockito **22**

知识点 22

题目 24

Part 6 REST API：Controller、参数校验、统一错误处理 **26**

知识点 26

题目 29

全部完成 30

附录 A 提示 **32**

Part 1 32

Part 2 32

Part 3 32

Part 4 33

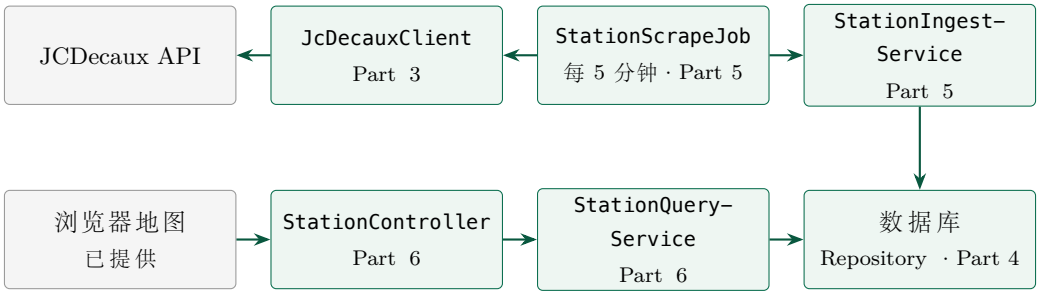
Part 5 33

Part 6 33

开始之前

这本练习册带你把 Dublin Bikes 的“车站”功能用 Spring Boot 重写一遍，边做题边学。题目格式模仿 OA：每一关分几个小题，每个小题有说明、示例和一组测试，测试全绿就算过关。

做完以后你会得到什么



一个能真正运行的后端：定时抓取都柏林 113 个站点的实时数据，存进数据库，再通过 REST API 提供给地图页面。

关卡

关卡	内容	测试	预计用时
Part 1	现代 Java：record、Stream、Optional（不涉及 Spring）	`Part1*`	1 小时
Part 2	Spring 核心：容器、Bean、依赖注入、配置绑定	`Part2*`	1 小时
Part 3	调用外部 API：RestClient、JSON 映射、错误处理	`Part3*`	1 小时
Part 4	数据库：JPA 实体、复合主键、Repository 查询	`Part4*`	1.5–2 小时
Part 5	事务和定时任务：`@Transactional`、`@Scheduled`、Mockito	`Part5*`	1 小时
Part 6	REST API：Controller、参数校验、统一错误处理	`Part6*`	1.5 小时

按顺序做。后面的关卡会用到前面写的代码，但每一关的测试是独立的：做完 Part 2 时，Part 1、2 的测试会全绿，Part 3 以后的测试还是红的，这是正常的。

准备

1. 用 IntelliJ IDEA 打开仓库的根目录（有 `pom.xml` 的那一层），等它自动导入 Maven 项目。
2. 确认 Project SDK 是 Java 25（File → Project Structure → Project）。
3. 跑一次全部测试，确认环境没问题。应该能编译，58 个测试里 57 个失败（只有 1 个检查 record 自动生成方法的测试会通过）：

```
./mvnw test
```

不需要安装 Maven，`mvnw` 会自动下载。不需要数据库，测试用的是内存数据库 H2。

做题流程

1. 读这一关的“知识点”，再读题目。
2. 在 `src/main/java` 里搜 `TODO 1.1` 这样的标记，找到要改的地方。
3. 跑这一关的测试：`./mvnw test -Dtest='Part1*'`。在 IntelliJ 里也可以直接点测试类旁边的绿色三角形，结果更直观。
4. 测试全绿后提交一次 commit，再进入下一关。

规则

- **不要改测试文件**（`src/test` 下面的所有文件）。测试就是题目要求。
- **不要改已经写好的类**：`NearbyStation`、`IngestResult`、`JcDecauxException`、`StationNotFoundException`、`StationResponse`、`NearbyStationResponse`，以及 Flyway 的 SQL 文件和 `application.yml`。
- 方法签名（方法名、参数、返回类型）不要改，否则测试编译不过。
- 卡住超过 20 分钟：先看附录里的提示，再问 AI 要思路。实在做不出来再看答案。

答案

`exercises/solutions/` 里是每个文件的参考答案，路径和 `src/main/java/ie/dublinbikes/` 一一对应。答案已经验证过：全部 58 个测试通过，应用能用真实的 JCDecaux 数据启动。**建议做完一关再看答案**，对比一下自己的写法哪里不同。

全部做完以后

```
./mvnw test                                  # 58 个测试全绿
SCRAPER_ENABLED=true ./mvnw spring-boot:run
```

几秒后打开 `http://localhost:8080` 就能看到地图上的 113 个站点。运行前先在仓库根目录建一个 `.env` 文件，写一行 `JCDECAUX_API_KEY=` 你的 key（可以从原项目的 `var.env` 里复制）。`.env` 已经在 `.gitignore` 里，不会被提交。

想连真实的 MySQL 而不是内存数据库（这条路径还没有实际测试过）：

```
docker compose up -d
SCRAPER_ENABLED=true ./mvnw spring-boot:run -Dspring-boot.run.profiles=mysql
```

用到的版本

Java 25、Spring Boot 4.1.2（Spring Framework 7、Hibernate 7、Jackson 3）。网上很多教程是 Spring Boot 2 或 3 的，大部分写法一样，个别包名不同，以这里的代码为准。

Part 1　现代 Java：record、Stream、Optional

这一关不涉及 Spring，目的是补上 Java 课上一般不讲、但 Spring 项目里到处都在用的写法。做完这一关，你会写出原 Python 项目里“找最近站点”功能的 Java 版本。

这一关

要改的文件

`station/domain/Station.java`：1.1

`station/domain/GeoUtils.java`：1.2

`station/domain/StationFinder.java`：1.3、1.4、1.5

（路径都相对于 `src/main/java/ie/dublinbikes/`）

运行测试　`./mvnw test -Dtest='Part1*'`

知识点

record：只用来装数据的类

以前写一个装数据的类，要写字段、构造函数、getter、`equals`、`hashCode`、`toString`，很长。record 一行就够了：

```
public record Book(String title, String author, double price) {}
```

编译器自动生成：

- 一个包含全部字段的构造函数：`new Book("Dune", "Herbert", 9.99)`
- 访问方法，**名字和字段一样，没有 get 前缀**：`book.title()`、`book.price()`
- `equals`、`hashCode`、`toString`：所有字段都相同的两个 record 用 `equals` 比较是相等的

record 的字段都是 `final` 的，创建以后不能修改。

紧凑构造函数（compact constructor）：想在创建时做校验，可以写一个不带参数列表的构造函数。参数会自动赋值给字段，你只需要写校验逻辑：

```
public record Range(int min, int max) {
    public Range {
        if (min > max) {
            throw new IllegalArgumentException("min must be <= max");
        }
    }
}
```

检查参数不能为 null，常用 `Objects.requireNonNull(value, " 错误信息")`，它会在 value 为 null 时抛 `NullPointerException`。

record 里也可以写普通方法：

```java
public record Book(String title, String author, double price) {
    public boolean isCheap() {
        return price < 10;
    }
}
```

字符串比较要防 null

```java
status.equals("OPEN")   // status 是 null 时抛 NullPointerException
"OPEN".equals(status)   // status 是 null 时返回 false，推荐这种写法
```

Python 数学函数对照

Python	Java
`math.radians(x)`	`Math.toRadians(x)`
`math.sin(x)`、`math.cos(x)`	`Math.sin(x)`、`Math.cos(x)`
`x ** 2`	`Math.pow(x, 2)` 或 `x * x`
`math.sqrt(x)`	`Math.sqrt(x)`
`math.atan2(y, x)`	`Math.atan2(y, x)`
`round(x)`	`Math.round(x)`，返回 `long`

Stream：用流水线处理集合

Python 里会用列表推导式和 `sorted`，Java 里用 Stream。比如“找出 Herbert 写的书，按价格从低到高，取前 2 本的书名”：

```java
List<String> titles = books.stream()                          // 1. 把 List 变成 Stream
        .filter(b -> b.author().equals("Herbert"))            // 2. 过滤：只留下满足条件的
        .sorted(Comparator.comparingDouble(Book::price))      // 3. 排序：按 price 从小到大
        .limit(2)                                             // 4. 只取前 2 个
        .map(Book::title)                                     // 5. 转换：Book -> String
        .toList();                                            // 6. 收集成 List
```

几个要点：

- `b -> b.author().equals("Herbert")` 是 **lambda**，相当于 Python 的 `lambda b: b.author == "Herbert"`。
- `Book::price` 是**方法引用**，是 `b -> b.price()` 的简写。
- `Comparator.comparingDouble(...)` 按 double 排序；按 long 排序用 `Comparator.comparingLong(...)`，按 int 用 `comparingInt`。
- `map` 可以把每个元素转成另一种类型，比如把 `Book` 转成一个新的 record。
- `.toList()` 返回的 List 不能修改。Stream 不会改动原来的 List。
- `sorted` 是**稳定排序**：排序依据相同的两个元素，保持原来的先后顺序。

Optional：明确表示“可能没有”

如果一个方法可能找不到结果，返回 `null` 很容易让调用方忘记判断，然后在别处抛 `NullPointerException`。返回 `Optional` 等于在类型上明确写着“可能为空”：

```
Optional<Book> cheapest = books.stream()
        .filter(Book::isCheap)
        .findFirst();              // findFirst() 返回的就是 Optional

cheapest.isPresent();              // 有没有值
cheapest.orElse(null);             // 有值返回值，没值返回 null
cheapest.orElseThrow();            // 有值返回值，没值抛异常
```

题目

1.1　Station record

`Station` 的字段已经定义好了。你要补两样东西：

a) 紧凑构造函数，校验：

- `name` 不能是 null，否则抛 `NullPointerException`
- `availableBikes`、`availableStands`、`totalStands` 都不能小于 0，否则抛 `IllegalArgumentException`

b) `isOpen()`：`status` 等于 `"OPEN"` 时返回 true。`status` 可能是 null，这时返回 false。

1.2　haversine 距离

把原项目 `main_project/journey/services.py` 里的 `haversine` 改写成 Java，实现 `GeoUtils.haversineMeters`，返回两个经纬度之间的距离（米）。地球半径用类里已经定义好的 `EARTH_RADIUS_METERS`。

```
def haversine(lat1, lng1, lat2, lng2) -> float:
    R = 6371000
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    d_phi = math.radians(lat2 - lat1)
    d_lambda = math.radians(lng2 - lng1)
    a = math.sin(d_phi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(
    ↪ d_lambda / 2) ** 2
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
```

Example

```
haversineMeters(53.3498, -6.2603, 53.349013, -6.260311)  ->  87.513...
haversineMeters(53.3498, -6.2603, 51.5074, -0.1278)      ->  463311.06...（都柏林
    ↪ 到伦敦）
```

1.3 findNearest：基本版

实现 `StationFinder.findNearest(lat, lng, stations, requireBikes, requireStands, limit)`。这一题先**忽略** `requireBikes` 和 `requireStands` 两个参数。

1. 对每个站点，计算它到 `(lat, lng)` 的距离，用 `Math.round` 四舍五入成整数米。
2. 包装成 `NearbyStation(station, distanceMeters)`（这个 record 已经写好了）。
3. 按距离从近到远排序；距离相同时保持原来的顺序。
4. 只返回前 `limit` 个。

要求用 Stream 写，不要用 for 循环。

Constraints

- `limit < 0` 时抛 `IllegalArgumentException`。
- `limit` 大于站点数时返回全部站点；`limit == 0` 时返回空列表。
- 不能修改传进来的 `stations`。

Example Input

查询点是 O'Connell Street 的 Spire：`lat = 53.3498, lng = -6.2603, limit = 3`。

number	name	lat	lng	status	bikes	stands
42	SMITHFIELD NORTH	53.349562	-6.278198	OPEN	3	27
5	CHARLEMONT PLACE	53.330662	-6.260177	OPEN	10	30
33	PRINCES STREET	53.349013	-6.260311	OPEN	0	20
2	BLESSINGTON STREET	53.356769	-6.26814	CLOSED	7	13
10	DAME STREET	53.344007	-6.266802	OPEN	5	0

Example Output

```
[NearbyStation(33, 88), NearbyStation(10, 775), NearbyStation(2, 933)]
```

全部 5 个站点的距离：33 → 88 米，10 → 775 米，2 → 933 米，42 → 1188 米，5 → 2128 米。

1.4 findNearest：过滤条件

在 1.3 的基础上加上两个过滤条件：

- `requireBikes == true`：排除 `availableBikes == 0` 的站点（用户要借车）。
- `requireStands == true`：排除 `availableStands == 0` 的站点（用户要还车）。
- **新需求**：只要打开了任意一个过滤条件，就同时排除没有开放（`!isOpen()`）的站点，因为关闭的站点既不能借也不能还。两个条件都没打开时，关闭的站点照常返回。

Example（同样的 5 个站点，limit = 10）

```
requireBikes=true,  requireStands=false  ->  [10, 42, 5]
requireBikes=false, requireStands=true   ->  [33, 42, 5]
requireBikes=true,  requireStands=true   ->  [42, 5]
```

1.5　findByNumber

在列表里找编号为 `number` 的站点，找到返回 `Optional.of(站点)`，找不到返回 `Optional.empty()`。**不要返回 null。**

想一想（面试常问）

1. record 为什么设计成不可修改的？在多线程环境下有什么好处？
2. 什么时候用 Stream，什么时候用普通 for 循环更合适？
3. 方法返回 `Optional` 和返回 `null` 相比，好处是什么？为什么不建议把 `Optional` 用作类的字段或方法参数？

Part 2　Spring 核心：容器、Bean、依赖注入、配置

这一关学 Spring 最核心的概念。后面几关的每一个类，都是按这一关的方式交给 Spring 管理的。

这一关

要改的文件

`config/AppProperties.java`：2.1、2.2

`config/AppConfig.java`：2.1、2.3

`station/domain/StationFinder.java`：2.4

运行测试　`./mvnw test -Dtest='Part2*'`

知识点

Spring 容器和 Bean

在普通 Java 程序里，你自己 `new` 对象，自己把对象传给需要它的地方：

```
BookRepository repo = new BookRepository(dataSource);
BookService service = new BookService(repo);
BookController controller = new BookController(service);
```

项目一大，这种“谁 new 谁、谁传给谁”的代码会非常多。Spring 的做法是：**你只声明有哪些类、每个类需要什么，Spring 负责创建对象并把它们连起来。**

- 由 Spring 创建和管理的对象叫 **Bean**。
- 管理所有 Bean 的东西叫**容器**（ApplicationContext）。
- 默认每个 Bean 只创建一个实例（单例），所有用到它的地方共享同一个。

怎么把一个类注册成 Bean

方法一：在类上加注解，让 Spring 扫描到。

```
@Component
public class PriceCalculator { ... }
```

启动类 `DublinBikesSpringApplication` 上的 `@SpringBootApplication` 会扫描它所在的包（`ie.dublinbikes`）以及所有子包，找到带这类注解的类，自动创建 Bean。

`@Component` 是通用的。还有几个意思更具体的，效果一样，只是让代码更好读：

注解	用在
`@Component`	通用组件
`@Service`	业务逻辑层
`@Repository`	数据访问层
`@RestController`	提供 REST 接口的类（Part 6）

方法二：在配置类里写一个方法，返回值就是 Bean。有些类不是你写的（比如 JDK 的 `Clock`），没办法在上面加注解，就用这种方式：

```
@Configuration                  // 表示这是一个配置类
public class MyConfig {

    @Bean                       // 这个方法的返回值会注册成 Bean
    public Random random() {
        return new Random(42);
    }
}
```

依赖注入（DI）：通过构造函数

一个 Bean 需要用到另一个 Bean 时，**在构造函数参数里声明就行**，Spring 创建它的时候会自动把需要的 Bean 传进来：

```
@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {   // Spring 自动传入 BookRepository 的
    ↪ Bean
        this.repository = repository;
    }
}
```

只有一个构造函数时，不需要加任何注解。这种写法叫**构造函数注入**，是官方推荐的方式。你会在网上看到另一种写法 `@Autowired private BookRepository repository;`（字段注入），不推荐，原因见本章最后的“想一想”。

配置文件绑定到 record

`src/main/resources/application.yml` 是应用的配置文件。打开看一下，里面有这样一段：

```
app:
  jcdecaux:
    api-key: ${JCDECAUX_API_KEY:}
  scraper:
    enabled: ${SCRAPER_ENABLED:false}
    interval: 5m
```

`${JCDECAUX_API_KEY:}` 的意思是：读取环境变量 `JCDECAUX_API_KEY`，没有的话用冒号后面的默认值（这里是空字符串）。

在代码里读这些配置，最好的方式是把它们**绑定到一个 record**：

```
@ConfigurationProperties(prefix = "library")
public record LibraryProperties(String name, int maxLoans, Duration loanPeriod) {}
```

```
library:
  name: Central
  max-loans: 5          # 配置里用 kebab-case，自动对应 Java 的 maxLoans
  loan-period: 14d      # 字符串自动转成 Duration（支持 30s、5m、2h、14d）
```

嵌套的配置对应嵌套的 record：`app.jcdecaux.api-key` 对应 `AppProperties` 里 `Jcdecaux` record 的 `apiKey`。

光写 `@ConfigurationProperties` 还不够，还要告诉 Spring "请把这个 record 注册成 Bean"。在某个 `@Configuration` 类上加：

```
@EnableConfigurationProperties(LibraryProperties.class)
```

之后任何 Bean 都可以在构造函数里要一个 `LibraryProperties`。

默认值：配置里没写的字段会是 null（基本类型是 0 或 false）。要给默认值，在 record 参数上加 `@DefaultValue`（包名 `org.springframework.boot.context.properties.bind`）：

```
public record LibraryProperties(
        @DefaultValue("Central") String name,
        @DefaultValue("5") int maxLoans) {}
```

嵌套 record 有个坑：如果配置文件里完全没有 `library.xxx` 这一段，嵌套的 record 本身会是 null。在**外层**的参数上加一个不带值的 `@DefaultValue`，就能让 Spring 用默认值创建它：

```
public record Outer(@DefaultValue Inner inner) {}
```

测试是怎么验证的

打开 `src/test/java/ie/dublinbikes/config/Part2SpringCoreTest.java` 看一下。`ApplicationContextRunner` 会启动一个只包含 `AppConfig` 的小容器，用 `withPropertyValues(...)` 模拟配置文件，然后检查容器里有没有你期望的 Bean、值对不对。

题目

2.1　配置绑定

让 `application.yml` 里 `app.*` 的配置能绑定到 `AppProperties`：

- `AppProperties` 对应前缀 `app`。
- `AppConfig` 是一个配置类，并且启用 `AppProperties` 的绑定。

Example

```
配置：app.jcdecaux.api-key=test-key
      app.jcdecaux.contract=lyon
      app.scraper.interval=30s

结果：props.jcdecaux().apiKey()   == "test-key"
      props.jcdecaux().contract() == "lyon"
      props.scraper().interval()  == Duration.ofSeconds(30)
```

2.2　默认值

什么都没配置时：

字段	默认值
`jcdecaux().apiKey()`	`null`（不需要默认值）
`jcdecaux().contract()`	`"dublin"`
`jcdecaux().baseUrl()`	`"https://api.jcdecaux.com/vls/v1"`
`scraper().enabled()`	`false`
`scraper().interval()`	5 分钟

而且 `jcdecaux()` 和 `scraper()` 本身不能是 null。

2.3　Clock Bean

在 `AppConfig` 里把 `clock()` 方法的返回值注册成 Bean，返回一个使用 `Europe/Dublin` 时区的系统时钟（看一下 `Clock.system(...)` 和 `ZoneId.of(...)`）。
为什么要把时钟做成 Bean，而不是在代码里直接用 `Instant.now()`？见本章最后的“想一想”。

2.4　组件扫描

让 Spring 能通过组件扫描自动发现 `StationFinder`。Part 6 的 `StationQueryService` 会在构造函数里要一个 `StationFinder`。

想一想（面试常问）

1. 为什么推荐构造函数注入，不推荐 `@Autowired` 字段注入？（提示：`final` 字段、不启动 Spring 能不能写单元测试、循环依赖）
2. Bean 默认是单例。如果一个单例 Bean 里有一个会被修改的字段，多个请求同时访问会出什么问题？
3. 代码里直接调用 `Instant.now()`，和注入一个 `Clock` 再调用 `Instant.now(clock)`，测试的时候有什么区别？
4. `@Component` 和 `@Bean` 分别适合什么场景？

Part 3　调用外部 API：RestClient 与 JSON 映射

这一关写一个 JCDecaux 的客户端：发 HTTP 请求，把返回的 JSON 转成 Java 对象，出错时抛出统一的异常。原 Python 项目里对应的是 `main_project/station/services.py` 的 `get_all_stations()` 和 `format_station()`。

这一关

要改的文件

`jcdecaux/JcDecauxStationDto.java`：3.1、3.2

`jcdecaux/JcDecauxClient.java`：3.3、3.4

运行测试　`./mvnw test -Dtest='Part3*'`

知识点

JSON 和 record 的转换（Jackson）

Spring Boot 自带 Jackson，会自动把 JSON 转成 record，前提是**名字对得上**：

```
{ "title": "Dune", "price": 9.99, "author": { "name": "Herbert" } }
```

```
public record Book(String title, double price, Author author) {
    public record Author(String name) {}      // 嵌套对象对应嵌套 record
}
```

JSON 里的字段名和 Java 不一样时（外部 API 经常用 snake_case），用 `@JsonProperty` 指定 JSON 里的名字：

```
public record Book(
        String title,
        @JsonProperty("page_count") int pageCount) {}
```

包名

`@JsonProperty` 的包名是 `com.fasterxml.jackson.annotation`。Spring Boot 4 用的是 Jackson 3，大部分类的包名改成了 `tools.jackson`，**但注解的包名没变**，IntelliJ 自动导入时注意别选错。

基本类型和包装类型：JSON 里某个字段可能是 `null` 时，Java 里要用包装类型（`Long`、`Integer`），因为 `long`、`int` 不能是 null。看一下测试用的假数据 `src/test/resources/fixtures/jcdecaux-stations.json`，第三个站点的 `last_update` 就是 `null`。

时间戳：JCDecaux 的 `last_update` 是毫秒时间戳（从 1970 年开始的毫秒数），用 `Instant.ofEpochMilli(long)` 转成 `Instant`。

DTO 和领域对象

`JcDecauxStationDto` 叫 **DTO**（Data Transfer Object）：它的结构完全按照 JCDecaux 的 JSON 来，只负责接收数据。我们自己的代码用的是 Part 1 写的 `Station`。

为什么要分开？如果 JCDecaux 改了字段名，只需要改 DTO 和转换方法，其他代码都不受影响。

RestClient：发 HTTP 请求

```
RestClient client = builder.baseUrl("https://api.example.com").build();

List<Book> books = client.get()                                    // GET 请求
        .uri(uri -> uri.path("/books")                             // 路径拼在 baseUrl 后面
                .queryParam("author", "Herbert")                   // ?author=Herbert
                .build())
        .retrieve()                                                // 发送请求
        .body(new ParameterizedTypeReference<List<Book>>() {});  // 响应体转成 List<Book>
```

- **为什么注入 `RestClient.Builder`，而不是直接 `RestClient.create()`？** Spring Boot 提供的 Builder 已经配好了 Jackson 等设置，测试时还能被 `MockRestServiceServer` 接管。
- **为什么要 `ParameterizedTypeReference`？** Java 的泛型在运行时会被擦除，`List<Book>.class` 这种写法不存在。如果只是转成单个对象，直接写 `.body(Book.class)` 就行；转成 `List<Book>` 这种带泛型的类型，就要用这个写法把完整的类型信息传进去。

错误处理

调用外部 API 有两类错误：

情况	RestClient 的默认行为
对方返回 4xx / 5xx	抛 `HttpClientErrorException` / `HttpServerErrorException`
网络问题：连不上、超时、连接被重置	抛 `ResourceAccessException`

这些异常都继承自 `RestClientException`。我们不希望调用方关心这些细节，所以统一转成自己的 `JcDecauxException`（已经写好了）。

处理 4xx / 5xx：在 `retrieve()` 之后加 `onStatus`：

```
.retrieve()
.onStatus(HttpStatusCode::isError, (request, response) -> {
    throw new MyApiException("API returned HTTP " + response.getStatusCode().value());
})
.body(...)
```

处理网络错误：用 try-catch 把整个调用包起来，捕获 `RestClientException`，转成自己的异常。**一定要把原来的异常作为 cause 传进去**（`new MyApiException(message, e)`），否则排查问题时就看不到真正的原因了。

想一想：`onStatus` 里抛的 `JcDecauxException` 会不会又被外面的 catch 抓住？（不会，因为 `JcDecauxException` 不是 `RestClientException` 的子类。）

测试是怎么验证的

`Part3JcDecauxClientTest` 用了 `@RestClientTest(JcDecauxClient.class)`：它只启动和 HTTP 客户端相关的部分，并且把 `RestClient.Builder` 接到一个 `MockRestServiceServer` 上。测试里写“期望收到某个 URL 的请求，返回某个 JSON”，你的代码发出的请求不会真的到达 JCDecaux。

题目

3.1　JSON 字段映射

JCDecaux 返回的一条数据长这样（完整的假数据在 `src/test/resources/fixtures/jcdecaux-stations.json`）：

```
{
  "number": 42,
  "contract_name": "dublin",
  "name": "SMITHFIELD NORTH",
  "address": "Smithfield North",
  "position": { "lat": 53.349562, "lng": -6.278198 },
  "banking": false,
  "bonus": false,
  "bike_stands": 30,
  "available_bike_stands": 27,
  "available_bikes": 3,
  "status": "OPEN",
  "last_update": 1761400800000
}
```

`JcDecauxStationDto` 的字段已经声明好了，但有 5 个字段的 Java 名字和 JSON 名字不一样。给它们加上正确的注解。

3.2　转换方法

a) **`lastUpdateInstant()`**：把 `lastUpdate`（毫秒时间戳）转成 `Instant`；`lastUpdate` 为 null 时返回 null。

Example

```
lastUpdate = 1761400800000  ->  2025-10-25T14:00:00Z
lastUpdate = null           ->  null
```

b) **`toStation()`**：转成领域对象 `Station`。注意对应关系：

Station	JcDecauxStationDto
availableBikes	availableBikes
availableStands	availableBikeStands
totalStands	bikeStands
lat、lng	position.lat、position.lng

3.3　发请求

a) `JcDecauxClient` 要交给 Spring 管理，在构造函数里用注入的 `RestClient.Builder` 创建 `RestClient`，`baseUrl` 用配置里的 `baseUrl`。
b) 实现 `fetchStations()`：

```
GET {baseUrl}/stations?contract={contract}&apiKey={apiKey}
```

查询参数的顺序必须是先 `contract` 再 `apiKey`（测试会比较完整的 URL）。返回的 JSON 数组转成 `List<JcDecauxStationDto>`。

Example

```
baseUrl  = https://jcdecaux.test/vls/v1,  contract = dublin,  apiKey = test-key
请求 URL = https://jcdecaux.test/vls/v1/stations?contract=dublin&apiKey=test-key
```

3.4　错误处理

- 对方返回 4xx 或 5xx：抛 `JcDecauxException`，消息里要包含状态码（比如 `"JCDecaux returned HTTP 403"`）。
- 网络错误：抛 `JcDecauxException`，并把原始异常作为 cause。

想一想（面试常问）

1. 外部 API 很慢或者挂了，我们的应用会怎样？应该给 RestClient 设置什么？（提示：连接超时、读取超时）
2. 如果 JCDecaux 偶尔返回 503，重试是好主意吗？重试几次、间隔多久？什么样的请求不应该重试？
3. API key 放在 URL 的查询参数里有什么风险？（提示：日志、代理服务器）

Part 4　数据库：JPA 实体、复合主键、Repository

这一关把站点和可用量存进数据库。这是 Spring Boot 里内容最多、也最容易卡住的一块，慢慢来。

这一关

要改的文件

`station/persistence/StationEntity.java`：4.1

`station/persistence/AvailabilityId.java`、`AvailabilityEntity.java`：4.2

`station/persistence/StationRepository.java`：4.3

`station/persistence/AvailabilityRepository.java`：4.4、4.5

运行测试　`./mvnw test -Dtest='Part4*'`

知识点

几个名字的关系

- **JPA**（Jakarta Persistence API）：Java 官方定义的一套“把对象存进关系数据库”的标准，主要是一组注解和接口，包名是 `jakarta.persistence`。
- **Hibernate**：JPA 最常用的实现。真正生成 SQL、执行查询的是它。
- **Spring Data JPA**：在 JPA 上面再包一层，让你只写接口、不写实现就能完成大部分查询。

表结构由 Flyway 管理

先打开 `src/main/resources/db/migration/V1__create_station_tables.sql` 看一下两张表：

```
CREATE TABLE station (
    number      INT          NOT NULL,
    name        VARCHAR(255) NOT NULL,
    address     VARCHAR(255),
    lat         DOUBLE       NOT NULL,
    lng         DOUBLE       NOT NULL,
    banking     BOOLEAN      NOT NULL,
    bike_stands INT          NOT NULL,
    status      VARCHAR(32),
    PRIMARY KEY (number)
);

CREATE TABLE availability (
    number                INT          NOT NULL,
    last_update           TIMESTAMP(6) NOT NULL,
    available_bikes       INT          NOT NULL,
    available_bike_stands INT          NOT NULL,
    status                VARCHAR(32),
```

```
    PRIMARY KEY (number, last_update),
    CONSTRAINT fk_availability_station FOREIGN KEY (number) REFERENCES station (number)
);
```

应用启动时，Flyway 会按版本号依次执行 `db/migration` 下还没执行过的脚本。表结构因此有了版本记录，和代码一起提交到 git。

`application.yml` 里设置了 `spring.jpa.hibernate.ddl-auto: validate`，意思是：Hibernate 不建表、不改表，只**检查**你的实体类和数据库里的表是否对得上，对不上就启动失败。所以你的注解写错了，测试会直接告诉你哪一列有问题。

实体（Entity）：一个类对应一张表

```java
@Entity                                 // 这是一个实体
@Table(name = "book")                   // 对应 book 表（不写的话默认用类名）
public class BookEntity {

    @Id                                 // 主键
    private long isbn;

    @Column(nullable = false)           // 这一列不允许为空
    private String title;

    @Column(name = "page_count")        // 指定列名
    private int pageCount;

    private String author;              // 不加注解也会映射，列名默认按字段名

    protected BookEntity() {}           // JPA 要求有一个无参构造函数
    ...
}
```

- 注解都在 `jakarta.persistence` 包里。
- Spring Boot 默认会把 camelCase 字段名转成 snake_case 列名（`pageCount` → `page_count`），所以 `@Column(name = ...)` 很多时候可以省略。这一关要求你显式写出来，练习一下。
- JPA 通过反射创建对象，所以需要一个无参构造函数。写成 `protected` 是为了不让业务代码随便用它创建出不完整的对象。

复合主键

`availability` 表的主键有两列。JPA 的做法是：把两列包装成一个“可嵌入的值对象”，再把它作为实体的主键：

```java
@Embeddable                                         // 可以嵌入到实体里
public record LoanId(long isbn, @Column(name = "loan_date") LocalDate loanDate)
        implements Serializable {}                  // JPA 规范要求主键类可序列化

@Entity
```

```
public class LoanEntity {
    @EmbeddedId                                    // 这个字段是复合主键
    private LoanId id;
    ...
}
```

主键类还必须正确实现 `equals` 和 `hashCode`，用 record 的话这两个方法是自动生成的。

注意

Hibernate 7 其实很宽松：只要字段上有 `@EmbeddedId`，主键类上不加 `@Embeddable` 也能运行，测试也发现不了。但 JPA 规范要求加，换一个 JPA 实现可能就不行了，所以还是要写上。

Repository：只写接口

```
public interface BookRepository extends JpaRepository<BookEntity, Long> {
    //                                          实体类型     主键类型
}
```

继承 `JpaRepository` 之后，不用写任何代码就有了 `save`、`saveAll`、`findById`、`findAll`、`count`、`existsById`、`deleteById` 等方法。Spring 在启动时自动生成实现类。

派生查询（derived query）：按规则给方法起名，Spring 会根据方法名自动生成查询：

```
List<BookEntity> findByAuthor(String author);
// -> select ... where author = ?

List<BookEntity> findByAuthorOrderByTitleAsc(String author);
// -> select ... where author = ? order by title asc

Optional<BookEntity> findFirstByAuthorOrderByPublishedDesc(String author);
// -> 某个作者最新出版的一本书
```

方法名里的属性名是**实体的字段名**，不是表的列名。属性在嵌入对象里面时，把路径连起来写：主键 `id` 里面的 `isbn` 写成 `IdIsbn`，比如 `findByIdIsbn(long isbn)`。

接口里的 default 方法：Java 8 起，接口里可以写有方法体的 `default` 方法。这一关里，测试调用的是 `findOpenStations()`、`findLatest(...)` 这些名字好读的方法，你在里面调用自己声明的派生查询。

自己写查询：@Query 和 JPQL

方法名表达不了的查询，用 `@Query` 写 **JPQL**。JPQL 长得像 SQL，但写的是**实体名和字段名**，不是表名和列名：

```
@Query("select b from BookEntity b where b.price > :minPrice")
List<BookEntity> findExpensive(@Param("minPrice") double minPrice);
```

一个常见需求是“每组取最新的一条”，比如每个作者最新出版的书。可以用子查询：

```
@Query("""
        select b from BookEntity b
        where b.published = (
            select max(b2.published) from BookEntity b2
            where b2.author = b.author)
        """)
List<BookEntity> findNewestBookOfEachAuthor();
```

（""" 是 Java 的文本块，可以写多行字符串。）

测试是怎么验证的

`@DataJpaTest` 只启动数据库相关的部分：内存数据库 H2、Flyway、JPA 和 Repository。每个测试方法结束后会**自动回滚**，所以测试之间的数据互不影响。

测试里的 `flushAndClear()`：`flush` 把还没写进数据库的修改真正执行成 SQL，`clear` 清空 JPA 的一级缓存。这样接下来的查询一定会真的去查数据库，而不是直接从内存里拿刚才 save 的那个对象。

题目

4.1　StationEntity

给 `StationEntity` 加上注解，对应 `station` 表：

- 类是一个实体，对应 `station` 表。
- `number` 是主键。
- `name` 列不允许为空。
- `bikeStands` 显式指定列名 `bike_stands`。

Example

`save` 一个站点，`flushAndClear` 之后用 `findById(42)` 读出来，所有字段都和存进去时一样。

4.2　AvailabilityEntity 和复合主键

- `AvailabilityId` 是可嵌入的值对象；`lastUpdate` 对应 `last_update` 列。
- `AvailabilityEntity` 是实体，对应 `availability` 表；`id` 字段是复合主键；`availableBikes`、`availableBikeStands` 分别对应 `available_bikes`、`available_bike_stands` 列。

Example

```
save (42, 08:00) bikes=3
save (42, 08:05) bikes=5
findById(new AavailabilityId(42, 08:05))  ->  bikes=5
count()  ->  2

再 save (42, 08:00) bikes=4      // 主键相同
count()  ->  2                   // 不会多出一行，而是更新原来那行
```

4.3　查询开放的站点

在 StationRepository 里：
a) 声明一个派生查询方法：按 status 查找，结果按 name 升序。
b) findOpenStations() 调用它，status 传 "OPEN"。

Example

```
站点：CHARLEMONT PLACE (OPEN), BLESSINGTON STREET (CLOSED), ANNESLEY BRIDGE (OPEN
    ↪ )
findOpenStations()  ->  [ANNESLEY BRIDGE, CHARLEMONT PLACE]
```

4.4　某个站点最新的一条记录

在 AvailabilityRepository 里：
a) 声明一个派生查询方法：按主键里的 number 查找，按主键里的 lastUpdate 倒序，只取第一条，返回 Optional。
b) findLatest(number) 调用它。

Example

```
站点 42 有 08:00、08:05、08:10 三条记录
findLatest(42)  ->  08:10 那一条
findLatest(5)   ->  Optional.empty()（站点 5 没有记录）
```

4.5　每个站点最新的一条记录

把 findLatestForAllStations() 改成用 @Query 实现：**删掉 default 关键字和方法体**，在方法上加 @Query，写一条 JPQL。

Example

```
42: 08:00, 08:10
5:  08:00, 08:05
findLatestForAllStations()  ->  [(42, 08:10), (5, 08:05)]（顺序不限）
```

Part 6 会用这个查询：一次查出所有站点的最新数据，而不是每个站点查一次。

常见报错

报错信息里有	原因
`Not a managed type: class ...StationEntity`	类上没加 `@Entity`
`Schema validation: missing column [xxx] in table [yyy]`	字段对应的列名和表里的不一样
`Schema validation: missing table [xxx]`	`@Table(name = ...)` 写错了
`Entity '...' has no identifier`	实体里没有标 `@Id` 或 `@EmbeddedId` 的字段
`No property 'xxx' found for type 'yyy'`	派生查询的方法名里，属性名拼错了
`Could not resolve root entity 'xxx'`	`@Query` 的 JPQL 里写的是表名，应该写实体类名

报错信息通常很长，而且同一个测试类里只要有一个测试启动失败，其他测试都会报 `ApplicationContext failure threshold exceeded`。**只看第一个失败的测试**，找 `Caused by:` 开头的最后一行，那里一般是真正的原因。

想一想（面试常问）

1. 生产环境为什么用 Flyway 管理表结构，而不是让 Hibernate 自动建表（`ddl-auto: update`）？
2. 什么是 N+1 查询问题？如果 Part 6 里对每个站点各调用一次 `findLatest`，会发出多少条 SQL？
3. `save()` 一个主键已经存在的实体时，JPA 是怎么决定执行 INSERT 还是 UPDATE 的？
4. 为什么实体类不用 record？（提示：JPA 需要无参构造函数，而且要能修改字段）

Part 5　事务和定时任务：@Transactional、@Scheduled、Mockito

这一关把前面的部分串起来：每隔 5 分钟调用 JCDecaux（Part 3），把结果写进数据库（Part 4）。它替代的是原项目里 3 个 `while True: sleep(300)` 的 Python 爬虫脚本。

这一关

要改的文件

`station/ingest/StationIngestService.java`：5.1
`station/ingest/StationScrapeJob.java`：5.2、5.3
`config/AppConfig.java`：5.3

运行测试　`./mvnw test -Dtest='Part5*'`

知识点

事务（Transaction）

一次抓取要写 113 个站点和 113 条可用量记录。如果写到一半出错了，我们希望**要么全部写进去，要么全部不写**，不要留下一半新一半旧的数据。数据库的"事务"就是做这个的。

在 Spring 里，给方法加上 `@Transactional`：

```
@Service
public class TransferService {

    @Transactional
    public void transfer(long from, long to, double amount) {
        accounts.withdraw(from, amount);
        accounts.deposit(to, amount);  // 这里抛 RuntimeException 的话，上面的取款也会被撤
    ↪ 销
    }
}
```

`@Transactional` 的包名是 `org.springframework.transaction.annotation`。

- 方法正常结束：提交（commit）。
- 方法抛出 `RuntimeException`：回滚（rollback）。注意，受检异常（checked exception）默认**不会**回滚。
- `@Transactional(readOnly = true)`：只读事务，Part 6 会用到。

@Transactional 是怎么生效的：代理

Spring 不会修改你的类。它在创建 Bean 时，生成一个**代理对象**包在你的对象外面，别的 Bean 注入到的其实是这个代理：

```
调用方 --> 代理（开启事务）--> 你的对象.transfer(...) --> 代理（提交或回滚）
```

经典的坑：自调用

同一个类里，一个方法调用另一个带 `@Transactional` 的方法，事务不会生效，因为这是 `this.xxx()` 直接调用，没有经过代理。
这也是为什么这一关把“定时触发”（`StationScrapeJob`）和“写数据库”（`StationIngestService`）拆成两个类：`StationScrapeJob` 调用的是 `StationIngestService` 的代理，事务能正常生效。面试很喜欢问这个。

upsert 和幂等

upsert：有就更新，没有就插入。用 Part 4 的 Repository 可以这样写：

```
BookEntity book = bookRepository.findById(isbn)
        .orElseGet(() -> new BookEntity(isbn));   // 找不到就新建一个
book.updateDetails(...);
bookRepository.save(book);
```

`orElseGet` 和 `orElse` 的区别：`orElse(new X())` 不管有没有值都会先执行 `new X()`；`orElseGet(() -> new X())` 只在没有值的时候才执行。

幂等：同一个操作执行一次和执行多次，结果一样。JCDecaux 的数据不是每 5 分钟都会变，如果某个站点的 `last_update` 和上次一样，说明没有新数据，不应该再插一条。插入之前用 `existsById` 检查一下就行。

定时任务：@Scheduled

```
@Component
public class ReportJob {

    @Scheduled(fixedDelayString = "${report.interval}")   // 从配置里读间隔
    public void generate() { ... }
}
```

写法	含义
`fixedDelay` / `fixedDelayString`	上一次**执行完**以后，再等这么久执行下一次
`fixedRate` / `fixedRateString`	每隔这么久执行一次，不管上一次有没有执行完
`cron = "0 */5 * * * *"`	按 cron 表达式执行

`"${report.interval}"` 是占位符，Spring 会替换成配置里的值，`5m`、`30s` 这样的写法都可以。还要在某个 `@Configuration` 类上加 `@EnableScheduling`，定时任务才会真的运行。

日志

```
private static final Logger log = LoggerFactory.getLogger(ReportJob.class);

log.info("Generated {} reports in {} ms", count, elapsed);  // {} 按顺序替换成后面的参数
log.warn("Report failed: {}", e.getMessage());
```

用 `{}` 占位符，不要用字符串拼接：日志级别关闭时，占位符写法不会去拼字符串，性能更好。

Mockito：不启动 Spring 的单元测试

打开 `Part5ScrapeJobTest`。它没有启动 Spring，而是用 Mockito 造出假的依赖，再自己 `new` 出被测对象：

```
JcDecauxClient client = mock(JcDecauxClient.class);    // 假对象，所有方法默认什么都不做
when(client.fetchStations()).thenReturn(dtos);         // 规定：调用时返回 dtos
when(client.fetchStations()).thenThrow(new JcDecauxException("..."));  // 或者：调用时抛异
    ↪ 常

new StationScrapeJob(client, ingestService, props).run();

verify(ingestService).ingest(dtos);                    // 检查：ingest 被用 dtos 调用过一
    ↪ 次
verifyNoInteractions(client);                          // 检查：client 完全没被调用
```

能这样测试，前提是用了构造函数注入（Part 2）。这就是构造函数注入的一大好处。

题目

5.1　StationIngestService：写数据库

`StationIngestService` 是一个 Service，`ingest` 方法整体在一个事务里。
对每个 `JcDecauxStationDto`：

1. **站点 upsert**：用 `number` 查 `station` 表，有就更新，没有就新建，然后保存。站点信息用 `updateDetails(...)` 设置（name、address、lat、lng、banking、bikeStands、status）。
2. **可用量幂等插入**：如果 `lastUpdateInstant()` 不为 null，而且主键 `(number, lastUpdate)` 在数据库里还不存在，就插入一条 `AvailabilityEntity`（availableBikes、availableBikeStands、status）。

返回 `IngestResult(保存了几个站点, 新插入了几条可用量)`。

Example

```
第一次 ingest 3 个站点（last_update 都是 T1）      ->  IngestResult(3, 3)
同样的数据再 ingest 一次                          ->  IngestResult(3, 0)   // 幂等
ingest 站点 42，名字改了，last_update 变成 T2     ->  IngestResult(1, 1)   // 名字
    ↪ 被更新，多一条可用量
ingest 一个 last_update 为 null 的站点            ->  IngestResult(1, 0)
ingest 空列表                                     ->  IngestResult(0, 0)
```

5.2 StationScrapeJob：抓取流程

`StationScrapeJob` 交给 Spring 管理。`run()` 的逻辑：

1. 配置里 `app.scraper.enabled` 是 false：直接返回，什么都不调用。
2. 否则：调用 `client.fetchStations()`，把结果交给 `ingestService.ingest(...)`，然后打一条 info 日志，写清楚保存了几个站点、插入了几条记录。
3. 如果抛了 `JcDecauxException`：捕获，打一条 warn 日志，**不要往外抛**。定时任务里的异常没人处理，只会让日志里多一段很长的错误堆栈。

5.3 开启定时执行

- 让 `run()` 每隔 `app.scraper.interval` 执行一次，从**上一次执行完**开始计时。用占位符读取配置，冒号后面给一个默认值 `5m`。
- 在 `AppConfig` 上开启定时任务功能。

想一想（面试常问）

1. 如果把 `@Transactional` 加在 `StationScrapeJob.run()` 上，再在 `run()` 里调用同一个类的另一个方法写数据库，事务会生效吗？为什么？
2. 应用部署了 2 个实例，两个实例的定时任务会同时抓取、同时写库，会出什么问题？怎么解决？（提示：ShedLock、数据库唯一约束）
3. `@Transactional` 方法里抛出受检异常（比如 `IOException`），事务会回滚吗？
4. `fixedDelay` 和 `fixedRate` 有什么区别？如果一次抓取花了 6 分钟，间隔是 5 分钟，两种写法分别会怎样？

Part 6　REST API：Controller、参数校验、统一错误处理

最后一关：把数据库里的数据通过 REST API 提供出去。做完之后，`src/main/resources/static/index.html` 这个地图页面（已经写好了）就能显示所有站点。

这一关

要改的文件

`station/StationQueryService.java`：6.1
`station/web/StationController.java`：6.2、6.3
`web/GlobalExceptionHandler.java`：6.3

运行测试 `./mvnw test -Dtest='Part6*'`，最后 `./mvnw test -Dtest=FinalApplicationTest`

知识点

分层

```
HTTP 请求 --> Controller --> Service --> Repository --> 数据库
              (处理 HTTP)    (业务逻辑)   (数据访问)
```

- **Controller** 只负责 HTTP 相关的事：解析参数、校验、把结果转成 JSON、设置状态码。不写业务逻辑。
- **Service** 写业务逻辑，组合多个 Repository 的数据。不知道 HTTP 的存在。
- **Repository** 只负责读写数据库。

这一关有三种"站点"对象，各自负责一层：

类	层	作用
`StationEntity`	数据库	对应表结构
`Station`	业务	业务代码使用的对象（Part 1）
`StationResponse`	API	返回给前端的 JSON 结构（已经写好了）

为什么不直接把 Entity 返回给前端? 表结构一改，API 就跟着变，前端会出问题；而且 Entity 里可能有不该暴露的字段。

避免 N+1 查询

`listStations()` 要返回每个站点和它最新的可用量。最直接的写法是：

```
for (StationEntity s : stationRepository.findAll()) {
    availabilityRepository.findLatest(s.getNumber());   // 每个站点查一次
}
```

113 个站点就是 1 + 113 = 114 条 SQL，这就是 **N+1 问题**。更好的做法：用 Part 4.5 的查询，一次拿到所有站点的最新数据，放进一个 `Map< 站点编号, AvailabilityEntity>`，再逐个查 Map。这样总共只有 2 条 SQL。

```
Map<Long, BookEntity> byIsbn = books.stream()
        .collect(Collectors.toMap(b -> b.getIsbn(), b -> b));  // key 是 isbn，value 是对
    ↪ 象本身
byIsbn.get(isbn);                                              // 找不到时返回 null
```

`b -> b` 也可以写成 `Function.identity()`。按字段排序查全部：`stationRepository.findAll(Sort.by("number"))`。

Controller

```
@RestController                          // 返回值自动转成 JSON
@RequestMapping("/api/v1/books")         // 这个类里所有接口的公共路径
public class BookController {

    @GetMapping                          // GET /api/v1/books
    public List<BookResponse> list() { ... }

    @GetMapping("/{isbn}")               // GET /api/v1/books/123
    public BookResponse get(@PathVariable long isbn) { ... }

    @GetMapping("/search")               // GET /api/v1/books/search?q=dune&limit=5
    public List<BookResponse> search(
            @RequestParam String q,                                  // 必填，缺了返回 400
            @RequestParam(defaultValue = "10") int limit) { ... }  // 可选，有默认值
}
```

record 转成 JSON 时，字段名就是 record 的字段名（camelCase）：`StationResponse` 的 `availableBikes` 会变成 `"availableBikes": 3`。

参数校验

在参数上加 `jakarta.validation.constraints` 包里的注解，Spring 会在调用方法之前检查，不满足就直接返回 400，方法根本不会执行：

```
public List<BookResponse> search(
        @RequestParam @Size(min = 2) String q,
        @RequestParam(defaultValue = "10") @Min(1) @Max(50) int limit) { ... }
```

注解	用在
`@Min` / `@Max`	整数
`@DecimalMin("-90")` / `@DecimalMax("90")`	小数（值写成字符串）
`@NotBlank`、`@Size`、`@Pattern`	字符串

统一错误处理和 ProblemDetail

Controller 里抛出的异常，可以在一个全局的地方统一转成 HTTP 响应：

```
@RestControllerAdvice                                    // 对所有 Controller 生效
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(BookNotFoundException.class)       // 处理这种异常
    public ProblemDetail handleNotFound(BookNotFoundException e) {
        ProblemDetail problem =
                ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
        problem.setTitle("Book not found");
        problem.setProperty("isbn", e.getIsbn());        // 可以加自定义字段
        return problem;
    }
}
```

`ProblemDetail` 是 RFC 9457 定义的标准错误格式，返回的 Content-Type 是 `application/problem+json`：

```
{
  "title": "Book not found",
  "status": 404,
  "detail": "Book 123 does not exist",
  "instance": "/api/v1/books/123",
  "isbn": 123
}
```

继承 `ResponseEntityExceptionHandler` 的作用：Spring 自己抛的异常（缺少必填参数、参数校验失败、参数类型不对等）也会自动转成 400 的 ProblemDetail，不用一个个自己写。

测试是怎么验证的

- `Part6QueryServiceTest`：`@DataJpaTest` 加真实的内存数据库，测 Service。
- `Part6ControllerTest`：`@WebMvcTest(StationController.class)` 只启动 Web 层，不连数据库。`@MockitoBean` 把 `StationQueryService` 换成 Mockito 的假对象；`MockMvc` 模拟发 HTTP 请求，不用真的启动服务器；`jsonPath("$[0].availableBikes")` 检查 JSON 里的值。
- `FinalApplicationTest`：`@SpringBootTest` 启动整个应用，检查所有 Bean 都能正确创建、连在一起。前面任何一个类忘了加 `@Service`、`@Component`，这个测试都会失败。

题目

6.1 StationQueryService

`StationQueryService` 是一个 Service，类里所有方法都在**只读事务**里执行（类上加一次就行）。
a) **`listStations()`**：返回所有站点，按 `number` 升序。每个站点配上它最新的可用量，用已经写好的 `toStation(entity, availability)` 转换（`availability` 可以是 null）。**只能用 2 次查询**：一次查所有站点，一次用 `findLatestForAllStations()` 查所有最新可用量。

Example

```
站点：42、33、5
可用量：42 有 08:00（1 辆）和 08:05（3 辆）；33 有 08:05（0 辆）；5 没有记录

listStations()  ->  [5（0 辆, 0 空位）, 33（0 辆）, 42（3 辆, 27 空位）]
```

b) **`getStation(number)`**：返回单个站点，配上 `findLatest(number)` 的结果（可能没有）。站点不存在时抛 `StationNotFoundException`（已经写好了）。
c) **`findNearest(lat, lng, limit, requireBikes)`**：用 `listStations()` 拿到所有站点，再交给 Part 1 的 `StationFinder.findNearest`（`requireStands` 传 false）。

6.2 StationController

`StationController` 是一个 REST Controller，公共路径 `/api/v1/stations`。
a) **`GET /api/v1/stations`**：返回所有站点，每个用 `StationResponse.from(...)` 转换。

```
[
  {
    "number": 42, "name": "SMITHFIELD NORTH", "address": "Smithfield North",
    "lat": 53.349562, "lng": -6.278198, "status": "OPEN",
    "availableBikes": 3, "availableStands": 27, "totalStands": 30
  }
]
```

b) **`GET /api/v1/stations/{number}`**：返回单个站点，`number` 从路径里取。
c) **`GET /api/v1/stations/nearest`**：

参数	必填	默认值
`lat`	是	
`lng`	是	
`limit`	否	3
`requireBikes`	否	false

每个结果用 `NearbyStationResponse.from(...)` 转换。

Example

```
GET /api/v1/stations/nearest?lat=53.3498&lng=-6.2603
-> [{"number": 42, "name": "SMITHFIELD NORTH", ..., "distanceMeters": 1188}]
```

6.3　错误处理和参数校验

a) 站点不存在：完成 `GlobalExceptionHandler`，`StationNotFoundException` 转成 404 ProblemDetail：

```
GET /api/v1/stations/999

404, Content-Type: application/problem+json
{
  "title": "Station not found",
  "status": 404,
  "detail": "Station 999 does not exist",
  "stationNumber": 999,
  ...
}
```

b) 参数校验：`lat` 在 -90 到 90 之间，`lng` 在 -180 到 180 之间，`limit` 在 1 到 20 之间。不满足时返回 400 ProblemDetail，而且**不能调用** Service。

Example

```
GET /api/v1/stations/nearest?lat=100&lng=-6.26          ->  400
GET /api/v1/stations/nearest?lat=53.3&lng=-6.2&limit=50  ->  400
GET /api/v1/stations/nearest?lng=-6.26                   ->  400（缺少 lat）
```

全部完成

```
./mvnw test                                  # 58 个测试全绿
SCRAPER_ENABLED=true ./mvnw spring-boot:run
```

几秒钟后打开 `http://localhost:8080`，地图上应该有 113 个站点。在另一个终端试试这些请求：

```
curl localhost:8080/api/v1/stations/42
curl "localhost:8080/api/v1/stations/nearest?lat=53.3498&lng=-6.2603&requireBikes=true"
curl localhost:8080/api/v1/stations/99999
curl "localhost:8080/api/v1/stations/nearest?lat=200&lng=0"
```

想一想（面试常问）

1. Controller、Service、Repository 为什么要分层？如果把查询直接写在 Controller 里，会有什么问题？
2. `@Transactional(readOnly = true)` 有什么好处？
3. `@WebMvcTest` 和 `@SpringBootTest` 有什么区别？各自适合测什么？
4. 为什么 API 路径里要带版本号 `/v1`？要改成不兼容的格式时怎么办？
5. 现在每次请求 `/api/v1/stations` 都会查两次数据库，但数据 5 分钟才更新一次。怎么优化？（提示：缓存，下一阶段的内容）

附录 A　提示

卡住超过 20 分钟再来看这里。提示按题号排列，从思路到接近答案逐步具体。

Part 1

1.1

紧凑构造函数写在 record 的大括号里面，形式是 `public Station { ... }`，没有参数列表。里面可以直接用 `name`、`availableBikes` 这些参数名。

1.3

流水线的顺序：

```
stream()
  -> map(...)            转成 NearbyStation
  -> sorted(Comparator.comparingLong(NearbyStation::distanceMeters))
  -> limit(limit)
  -> toList()
```

`limit` 参数检查要放在 Stream 前面，因为 `Stream.limit` 收到负数时抛的异常信息不清楚。

1.4

可以连着写几个 `filter`。比如“如果打开了 requireBikes，就只保留有车的”，可以写成 `.filter(s -> !requireBikes || s.availableBikes() > 0)`。想一想为什么这样写是对的。

Part 2

2.1 报错 No qualifying bean of type AppProperties

说明容器里没有 `AppProperties` 这个 Bean。检查两处：`AppProperties` 上有没有 `@ConfigurationProperties(prefix = "app")`；`AppConfig` 上有没有 `@Configuration` 和 `@EnableConfigurationProperties(AppProperties.class)`。

2.2 嵌套对象是 null

在 `AppProperties` 的 `jcdecaux` 和 `scraper` 两个参数前面加 `@DefaultValue`。

Part 3

3.1 某个字段是 0 或 null

说明 Jackson 没找到对应的 JSON 字段。检查 `@JsonProperty` 里的名字是否和 JSON 完全一致，以及 import 的是不是 `com.fasterxml.jackson.annotation.JsonProperty`。

3.3 报错 No further requests expected 或 URL 不匹配

把报错信息里实际请求的 URL 和期望的 URL 逐个字符对比。常见问题：`path` 少了开头的 `/`，或者两个查询参数的顺序反了。

3.3 构造函数里 restClient 赋值

```
this.restClient = builder.baseUrl(props.baseUrl()).build();
```

Part 4

4.4 派生查询的方法名

拆开来看：`findFirst` + `By` + `IdNumber`（条件：id.number 等于参数）+ `OrderBy` + `IdLastUpdate` + `Desc`。

4.5 JPQL 写不出来

照着知识点里“每个作者最新的书”改：实体是 `AvailabilityEntity`，分组字段是 `a.id.number`，比较的字段是 `a.id.lastUpdate`。

Part 5

5.1 不知道从哪里开始

一个 for 循环，两个计数器 `stationsSaved` 和 `availabilityInserted`。站点部分照着知识点里的 upsert 写；可用量部分：先用 `dto.lastUpdateInstant()` 判断是否为 null（是 null 就 `continue`），再 `new AvailabilityId(dto.number(), lastUpdate)`，用 `existsById` 判断。

5.3 @Scheduled 的写法

```
@Scheduled(fixedDelayString = "${app.scraper.interval:5m}")
```

Part 6

6.1a

第一步，把 `findLatestForAllStations()` 的结果转成 Map：

```
Collectors.toMap(a -> a.getId().number(), Function.identity())
```

第二步：`stationRepository.findAll(Sort.by("number"))` 转成 Stream，`map` 里调用 `toStation(s, map.get(s.getNumber()))`。

6.1b

先找站点，找不到就抛异常；再找最新的可用量，找不到就用 null：

```
stationRepository.findById(number)
        .orElseThrow(() -> new StationNotFoundException(number));
availabilityRepository.findLatest(number).orElse(null);
```

6.3 404 测试过了，400 测试的 Content-Type 不对

检查 `GlobalExceptionHandler` 是否继承了 `ResponseEntityExceptionHandler`。

FinalApplicationTest 失败

找报错里 `Caused by:` 开头的最后一行，通常是 `No qualifying bean of type 'XXX'`，说明 XXX 这个类没有被 Spring 管理：检查它上面有没有 `@Service` 或 `@Component`。